

Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Automation and Applied Informatics

Comparison of Reinforcement Learning and Model Predictive Control for a Drone Equipped with a Manipulator

Scientific Students' Association Report

Author:

Tamás Epres

Advisor:

Péter Antal, PhD (HUN-REN SZTAKI)
Bence Szinyéri, (BME AUT)

2026

Contents

Kivonat i

Abstract ii

1 Introduction 1

2 System Modeling 3

2.1 Quadcopter Model . . . 3

2.2 Applied Manipulator . . . 5

2.3 Task Description . . . 7

3 Reinforcement Learning 9

3.1 Reinforcement Learning Formulation . . . 9

3.1.1 Observation . . . 10

3.1.2 Action . . . 10

3.1.3 Reward . . . 11

3.1.4 Episode . . . 12

3.1.5 Flat Agent . . . 12

3.2 Curriculum Learning . . . 12

3.2.1 Formulation . . . 13

3.2.2 Training with and without CL . . . 13

3.3 Hierarchical Reinforcement Learning . . . 14

3.3.1 Formulation . . . 15

3.3.2 Low-Level Observation . . . 16

3.3.3 Low-Level Action . . . 16

3.3.4 Low-Level Reward . . . 16

3.3.5 Low-Level Episode . . . 17

3.3.6 Low-Level Agent Results . . . 17

3.3.7 High-Level Policy . . . 21

3.3.8 High-Level Agent Results . . . 21

4 Model Predictive Control 22

4.1 Model Predictive Control Formulation . . . 22

Bibliography **24**

Kivonat

Az autonóm légi járművek alkalmazása mára szinte mindennapossá vált. Érdekes és ígéretes kutatási irány annak vizsgálata, hogyan használhatók ezek a járművek olyan feladatokra, amelyekben a drón fizikai kapcsolatba lép a környezetével, és kölcsönhatásba kerül vele.

A dolgozat egy ilyen rendszert vizsgál. Egy Bumblebee-alapú kvadkopter aljára spirálrobotok által ihletett, egyszerűsített, sok szabadsáfogkú, síkban mozgó manipulátor került. A drón feladata olyan trajektória előállítása, amelynek során egy előre nem ismert fizikai paraméterekkel (mérettel és tömeggel) rendelkező csomagot megfog, majd egyik helyről a másikra szállít. A rendszer vizsgálata MuJoCo-alapú szimulációs környezetben történik.

Az ilyen összetett feladatokat jellemzően kétféle megközelítéssel oldják meg. Az egyik az adatalapú irányítás. Ebben a szabályozó nem ismeri a rendszer fizikai paramétereit, hanem kizárólag a rendszer viselkedéséből és a korábbi trajektóriákból von le következtetéseket. A dolgozat ennek megfelelően egy Proximal Policy Optimization (PPO) alapú, hierarchikus megerősítéses tanulási (HRL) architektúrát mutat be. Ebben egy magas szintű ágens sebességreferenciát generál, amelyet egy előzetesen betanított, alacsony szintű ágens követ. A tanítást curriculum learning segíti (CL), amelynek során a feladat nehézsége és a csomag paramétereinek változatossága fokozatosan nő.

A másik paradigma egy identifikált matematikai modellre épül. Ennek segítségével előre jelezzük a rendszer viselkedését, a trajektóriát pedig optimalizációs eljárással tervezzük meg. A dolgozat ezt a módszert modellprediktív szabályozással (MPC) valósítja meg, amely a magas szintű ágens helyét veszi át, és az alacsony szintű ágensnek ad sebességreferenciát. Ehhez a drón és az azt irányító alacsony szintű ágens együttes viselkedését szimulációs adatok alapján egy reziduális neurális hálózattal (ResNet) identifikáltuk, és az MPC ezt a modellt használja predikcióra.

Végül a dolgozat összeveti a két megközelítés előnyeit és hátrányait, és javaslatot tesz egy hibrid módszerre, amely ötvözi a két módszer erősségeit. Ebben a megoldásban az RL-szabályozó kimeneteit egy optimalizáláson alapuló szabályozó szűri, és beavatkozik, ha a beavatkozójelek veszélyes állapotba vinnék a drónt.

Abstract

The use of autonomous aerial vehicles has become almost commonplace. An interesting and promising research direction is to investigate how these vehicles can be used for tasks in which the drone makes physical contact with its environment and interacts with it.

This thesis studies such a system. A simplified, two-dimensional manipulator inspired by spiral robots is mounted under a Bumblebee-based quadcopter. The task of the drone is to generate a trajectory along which it grasps a payload with unknown physical parameters (size and mass) and transports it from one location to another. The system is studied in a MuJoCo-based simulation environment.

Such complex tasks are typically solved with one of two approaches. The first is data-driven control, in which the controller does not know the physical parameters of the system and draws conclusions solely from the behavior of the system and from previous trajectories. Accordingly, the thesis presents a hierarchical reinforcement learning (HRL) architecture based on Proximal Policy Optimization (PPO). In this architecture, a high-level agent generates a velocity reference, which is tracked by a pre-trained low-level agent. Training is supported by curriculum learning (CL), during which the difficulty of the task and the variability of the payload parameters are gradually increased.

The second paradigm relies on an identified mathematical model, which is used to predict the behavior of the system, while the trajectory is planned by an optimization procedure. The thesis implements this approach with model predictive control (MPC), which takes the place of the high-level agent and provides the velocity reference to the low-level agent. To this end, the combined behavior of the drone and the low-level agent controlling it was identified from simulation data with a residual neural network (ResNet), and the MPC uses this model for prediction.

Finally, the thesis compares the advantages and disadvantages of the two approaches and proposes a hybrid method that combines their strengths. In this solution, the outputs of the RL controller are filtered by an optimization-based controller, which intervenes if the control inputs would drive the drone into an unsafe state.

Chapter 1

Introduction

The applications of unmanned aerial vehicles (UAVs) are solving increasingly complex tasks. Today, their use is no longer limited to passive tasks that require no physical contact with the environment, such as surveillance, video recording, or security and patrol missions. UAVs are now also deployed in tasks where the drone interacts physically with its surroundings. Examples include package delivery from a moving platform [2], the retrieval of flying vehicles [16], pipeline crack sealing [10], and even the collection of endangered plants from dangerous cliffs [7]. Due to the nature of these tasks, the drone is equipped with some kind of manipulator, which can range from a simple hook [2], through a multi-degree-of-freedom robotic arm [10], to a deformable, so-called soft robotic manipulator containing compliant segments [5].

This thesis is motivated by the ever faster and steadily growing demand for transportation. Supply chains require increasingly fast and inexpensive delivery, while safe transport and the protection of human life and property remain of paramount importance. An aerial manipulation system of this kind can offer a solution, as the drones appearing on the market are becoming cheaper and faster.

This thesis deals with a drone system in which a manipulator inspired by spiral robots [18] is mounted on the underside of the drone. The drone–manipulator system is described in detail in the *System Modelling* chapter. The task of the drone is to pick up a randomly placed package, whose mass and the radius of the cylinder used for grasping are not known in advance, and to deliver it to a target location. Throughout the task, safe flight and the maintenance of the yaw orientation are imposed as constraints, and the task must be completed within a given time limit.

Numerous approaches can be applied to realise such a trajectory planning algorithm. In recent years, learning-based (data-driven) control algorithms have become increasingly widespread, and their application has also extended to drone-mounted manipulator tasks [19]. These methods typically do not require an accurate description of the system; instead, they predict the system's behaviour based on collected data. They are generally robust to modelling errors and have very short execution times. On the other hand, their training is not trivial and is often a lengthy process, and safe control cannot be guaranteed. A data-driven reinforcement learning (RL) algorithm of this kind has been implemented in this thesis as well. To facilitate training, hierarchical RL was applied in combination with curriculum learning. The method is presented in detail in the *Reinforcement Learning* chapter.

Another approach is model predictive control (MPC). In this framework, a representation describing the system is available, based on which an optimisation problem can be formulated; solving it yields a suitable sequence of control inputs. This control strategy also has its advantages and disadvantages. Its advantage is that it can provide well-defined control inputs even in situations the drone has not previously encountered, while satisfying certain constraints. Its disadvantages are that, for higher-dimensional problems, its computation time may be longer, and it is sensitive to potential modelling errors. Such an MPC method has been applied in this thesis and is presented in the *Model Predictive Control* chapter, which also describes the system identification procedure used.

Finally, this thesis proposes a solution that combines the two methods so that each compensates for the shortcomings of the other. The two methods are used together in numerous applications [12]; this thesis investigates how MPC can be used to improve the safety of the RL controller. To this end, a safety filter similar to the one in [17] is presented. The filter is an MPC that supervises the control inputs proposed by the RL agent and overrides them whenever they would drive the system into an unsafe state. The method is described in detail in the *Safety Filter* section. Finally, the three approaches are compared in the *Comparison* section.

Chapter 2

System Modeling

This chapter presents the system to be controlled and the task to be performed. First, the applied quadcopter is introduced; its detailed description can be found in [1]. Next, the manipulator mounted on the bottom of the drone is presented, which is a spiral-robot-inspired manipulator [18]. Finally, the task to be performed, a transportation task, is described.

2.1 Quadcopter Model

The drone used in this work is the Bumblebee quadcopter of SZTAKI's AIMotionLab. Its modeling and control are described in detail in [1], which the following summary is based on.

The motion of the quadcopter is described in two coordinate frames (Figure 2.1): the inertial frame $\mathcal{F}^i$, fixed to the environment, and the body frame $\mathcal{F}^b$, attached to the center of mass of the drone. The configuration of the drone is given by the pair $(r, R) \in SE(3)$, where $r \in \mathbb{R}^3$ is its position in $\mathcal{F}^i$ and $R \in SO(3)$ is the rotation matrix from $\mathcal{F}^i$ to $\mathcal{F}^b$. The orientation can equivalently be described by the roll, pitch, and yaw angles $\eta = [\phi,\, \theta,\, \psi]^\top$, with

$$R(\phi, \theta, \psi) = \mathrm{Rot}_x(\phi)\,\mathrm{Rot}_y(\theta)\,\mathrm{Rot}_z(\psi). \tag{2.1}$$

Since Euler angles suffer from singularities, the rotation matrix representation is preferred for control design.

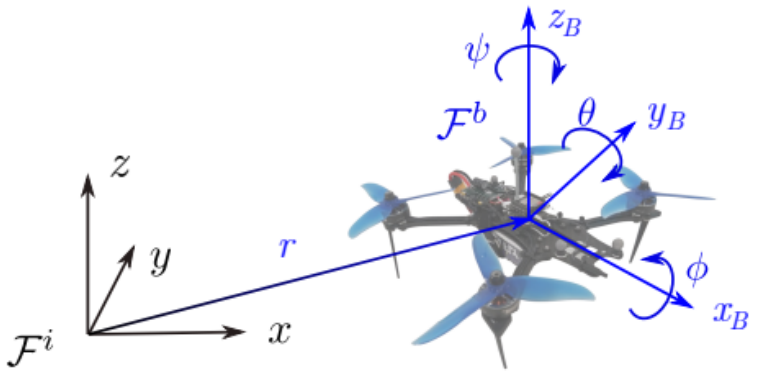


Figure 2.1: Inertial and body frames of the quadcopter, reproduced from [1, Fig. 3].

Similarly to [1, 9], the quadcopter is modeled as a rigid body with mass m and inertia matrix J, actuated by four propellers (Figure 2.2). Each propeller $i \in \{1, 2, 3, 4\}$ generates a thrust force and

a drag torque proportional to the square of its angular velocity ω_i:

$$f_i = c\omega_i^2, \qquad \tau_{z,i} = b\omega_i^2, \tag{2.2}$$

where c and b are the thrust and drag coefficients. The collective thrust and the torques in the body frame are

$$\begin{aligned} F &= f_1 + f_2 + f_3 + f_4, \\ \tau_x &= l_x(f_3 + f_4 - f_1 - f_2), \\ \tau_y &= l_y(f_2 + f_3 - f_1 - f_4), \\ \tau_z &= b(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2), \end{aligned} \tag{2.3}$$

where l_x and l_y are the distances of the propellers from the body axes. The dynamics follow the Newton–Euler equations

$$\begin{aligned} m\ddot{r} &= -mge_3 + FRe_3, \\ J\dot{\omega} &= \tau - \omega \times J\omega, \\ \dot{R} &= R\hat{\omega}, \end{aligned} \tag{2.4}$$

where $e_3 = [0, 0, 1]^\top$, ω is the angular velocity in $\mathcal{F}^b$, and $\hat{\omega}$ is its skew-symmetric matrix. The inputs of the model are thus the collective thrust F and the torque vector $\tau = [\tau_x, \tau_y, \tau_z]^\top$.

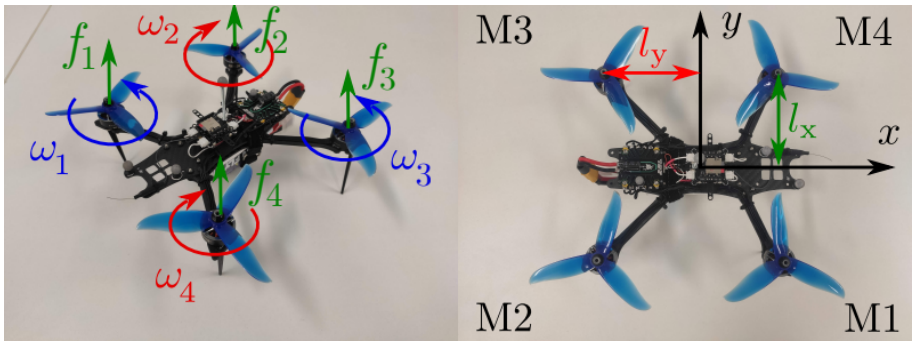


Figure 2.2: Propeller angular velocities and thrusts of the Bumblebee quadcopter, reproduced from [1].

The parameters of the model, measured in [1], are summarized in Table 2.1. The inertia matrix was computed from a detailed CAD model of the drone:

$$J = \begin{bmatrix} 1590.5 & -3.14 & -0.135 \\ -3.14 & 1481.3 & -49.18 \\ -0.135 & -49.18 & 2768.4 \end{bmatrix} \text{kg mm}^2. \tag{2.5}$$

Symbol	Description	Value
m	Mass of the quadcopter	$0.605\,\text{kg}$
J	Inertia matrix	see (2.5)
l_x, l_y	Propeller distances from the body axes	$0.125\,\text{m}$
c	Thrust coefficient	$9.3945 \times 10^{-7}\,\text{N s}^2/\text{rad}^2$
b	Drag coefficient	$\text{N m s}^2/\text{rad}^2$
g	Gravitational acceleration	$9.81\,\text{m/s}^2$

Table 2.1: Parameters of the quadcopter model.

2.2 Applied Manipulator

As mentioned in the introduction of this chapter, a spiral-robot-inspired manipulator is mounted on the bottom of the drone. The spiral robot is named after a pattern that occurs frequently in nature, the logarithmic spiral, which can be observed, for example, in the trunk of an elephant or the arm of an octopus. The motivation behind this manipulator is to create a device that, similarly to these solutions found in the animal kingdom, is capable of manipulating objects of various shapes and masses. As demonstrated in [18], such a robot can grasp objects with different geometries and masses, such as a tennis ball, an apple, or a pair of headphones, thereby achieving adaptivity. The authors of [18] also address the scalability of the robot, presenting both planar and spatial versions, and discuss its fabrication as well as contact detection.

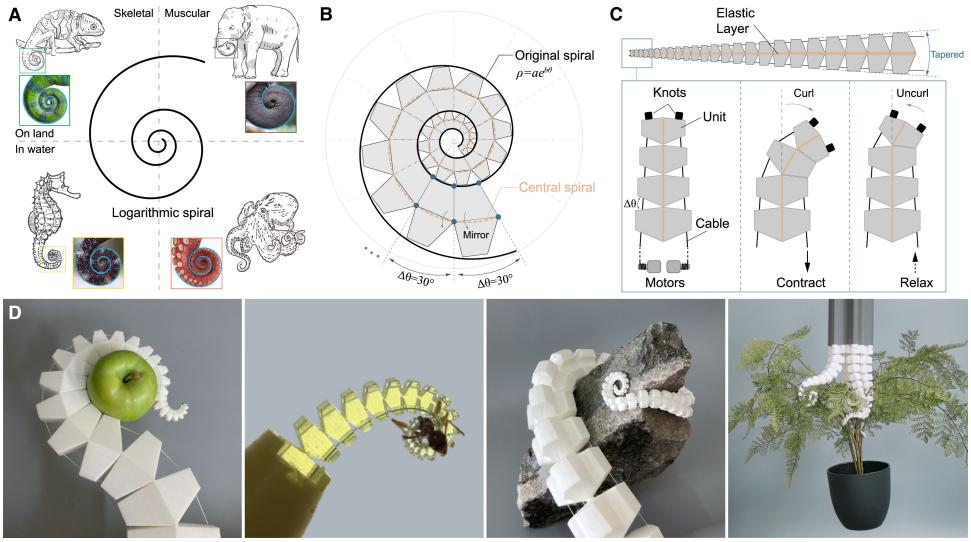


Figure 2.3: Applications and scalability of the spiral robot, reproduced from [18, Fig. 1]. (A) Logarithmic spirals in animal body parts (fitted spirals in blue). (B) Design: the discretized spiral is mirrored to form the robot units. (C) Elastic central layer and motor-driven cables that make the robot curl and uncurl. (D) 3D-printed robots grasping an apple, an ant, a rock, and a plant.

As shown in Figure 2.3(B) and (C), the spiral robot is not a soft manipulator. Instead, it consists of a series of rigid, 3D-printed segments that discretize the logarithmic spiral. The segments are connected by an elastic layer, which pulls the robot back to a straight configuration in its rest state. The robot is actuated by cables, which are routed along the edges of the segments and fixed to the last segment. When one of the cables is pulled, the robot curls in the corresponding direction, as illustrated in Figure 2.3(C).

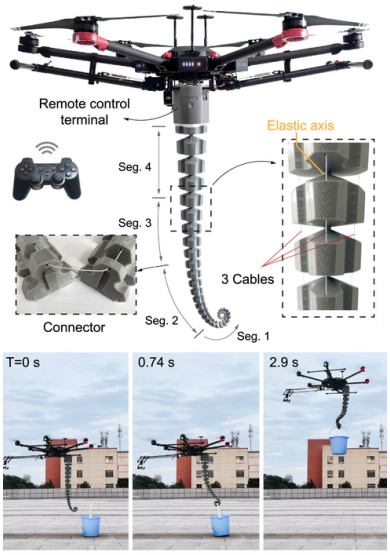


Figure 2.4: Application of the spiral robot on a drone, reproduced from [18, Fig. 5(E) and (F)]: the spiral robot mounted on the drone picks up a bucket.

This thesis can clearly be regarded as an extension of this work. Although [18] presents an application in which the spiral robot is mounted on a drone, as shown in Figure 2.4, it does not address the kinematic modeling or a rigorous control design of such a system. Instead, [18] focuses on the 3D printing, load-bearing capacity, contact sensing, and scalability of the spiral robot, together with a simple control approach.

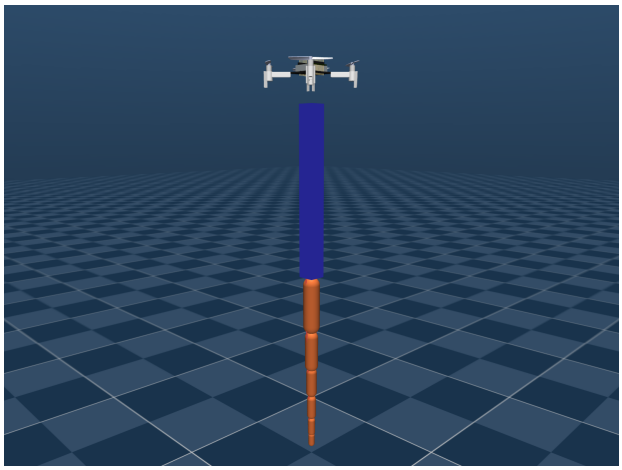


Figure 2.5: MuJoCo model of the applied drone and spiral robot system.

In this thesis, a simplified version of such a spiral robot is modeled in the MuJoCo simulation environment [15], as shown in Figure 2.5. In this version, the arm consists of only seven segments and can move in a single plane only.

It should be noted that the kinematic description of such a drone–manipulator system is far from trivial. For systems with many degrees of freedom, directly using the MuJoCo simulation environment within MPC is often a computational bottleneck, since the derivatives of the dynamics required by the optimizer have to be obtained by finite differencing (FD), as noted in [8]. The situation is further complicated by the question of how to describe the system while the drone is carrying a payload with unknown physical parameters. To address this, an identification procedure is presented in detail in the *Model Predictive Control* chapter.

2.3 Task Description

This section describes the expected behavior of the robot. As mentioned earlier, the task is a payload transportation task, structured as follows. The drone always starts from the same initial position with a random yaw orientation. A payload with unknown physical parameters is placed at a random position, and a goal position is also chosen randomly. The task of the drone is to pick up the payload and then fly to the goal position while carrying it. A typical setup is shown in Figure 2.6.

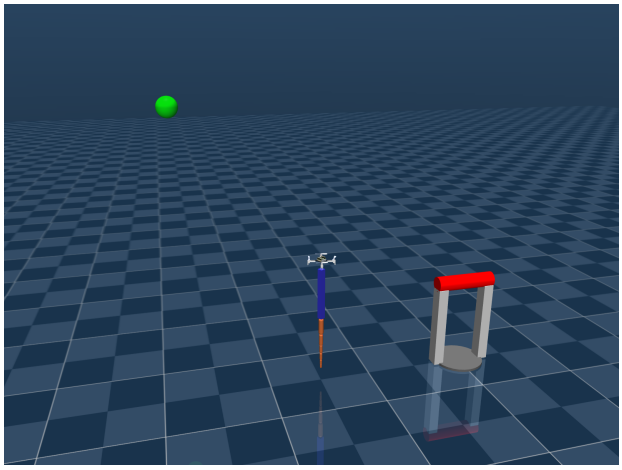


Figure 2.6: A typical scene setup with a randomly placed payload, a random goal position, and a random initial yaw angle of the drone.

The payload consists of a cylinder and two cuboids, which are attached to the same side of the cylinder at the two ends of its lateral surface. The parts of the payload are held together by a disk. The structure of the payload is shown in Figure 2.7.

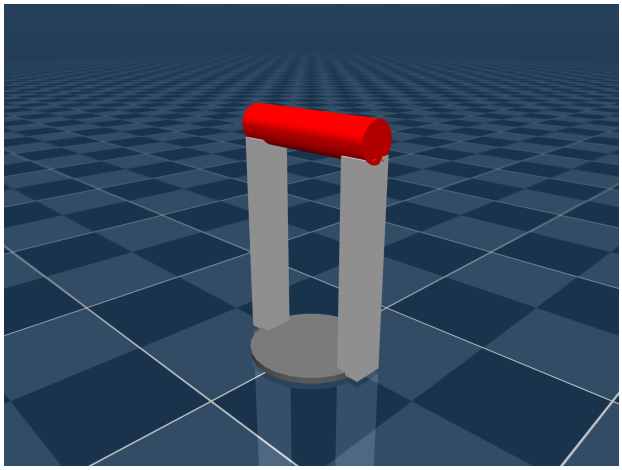


Figure 2.7: Structure of the payload. The robot grasps the red cylinder, whose radius is not known in advance, and neither is the mass of the disk at the bottom.

In each task, the radius of the cylinder, the mass of the disk, and the height of the payload are generated randomly. For a generic parameter X, we define

$$X \sim \mathcal{U}(X_{\min}, X_{\max}), \tag{2.6}$$

where $\mathcal{U}(a, b)$ denotes the uniform distribution on the interval $[a, b]$, and $X_{\min}$ and $X_{\max}$ are the lower and upper bounds of the parameter, respectively. Accordingly, the payload parameters are sampled as

$$R_{\text{payload}} \sim \mathcal{U}(R_{\min},\, R_{\max})\,, \tag{2.7a}$$
$$m_{\text{payload}} \sim \mathcal{U}(m_{\min},\, m_{\max})\,, \tag{2.7b}$$
$$z_{\text{payload}} \sim \mathcal{U}(z_{\min},\, z_{\max})\,, \tag{2.7c}$$

where R_{payload} is the radius of the cylinder, m_{payload} is the mass of the disk, and z_{payload} is the height of the payload.

As mentioned above, the horizontal position of the payload $(x_{\text{payload}}, y_{\text{payload}})$, the initial yaw angle of the drone ψ_0, and the goal position $(x_{\text{goal}}, y_{\text{goal}})$ are also generated randomly from uniform distributions, while the height of the goal position is fixed at $z_{\text{goal}} = 1\,\text{m}$. The bounds of all randomized quantities are summarized in Table 2.2.

Table 2.2: Bounds of the randomized task parameters.

Parameter	Minimum	Maximum	Unit
Cylinder radius R_{payload}	0.02	0.04	m
Disk mass m_{payload}	0.01	0.25	kg
Payload height z_{payload}	0.45	0.8	m
Payload position x_{payload}, y_{payload}	-1	1	m
Goal position x_{goal}, y_{goal}	-1.5	1.5	m
Initial yaw angle ψ_0	0	2π	rad

Chapter 3

Reinforcement Learning

This chapter presents a data-driven approach to solving the task, based on reinforcement learning (RL). The RL literature dates back several decades [14], but its major successes came only with the growth of available computational power, and numerous applications have emerged since.

RL is a natural choice for the problem described in the *Task Description* section. One of its key properties is that it does not require an exact mathematical model of the system; a simulator that accurately reproduces the system's behavior is sufficient. This is particularly advantageous here since, as discussed earlier, the dynamic model of the drone–manipulator system carrying a payload with unknown parameters is far from trivial, whereas an accurate MuJoCo model of it is available. Moreover, since the policy is trained in simulation, techniques such as domain randomization can be used to make it robust to modeling errors, which facilitates the transfer of the learned policy from simulation to the real system [20]. In this thesis, the widely used Proximal Policy Optimization (PPO) algorithm [13] is applied. The first part of the chapter introduces the RL methodology in the context of the thesis.

Despite these advantages, RL-based control also has several shortcomings. First, training can be very slow, depending on the complexity of the task. Second, in more complex environments, the agent may get stuck in local optima that do not produce the desired behavior. To address these issues, the second and third parts of this chapter introduce curriculum learning (CL) [11] and hierarchical reinforcement learning (HRL) [6].

3.1 Reinforcement Learning Formulation

The notation of [14] is adopted throughout this chapter. The interaction between the agent and its environment is modeled as an episodic Markov decision process (MDP)

$$m = (\mathcal{S},\, \mathcal{A},\, \mathcal{P},\, r,\, \rho_0,\, \mathcal{S}_f)\,, \tag{3.1}$$

where

- $\mathcal{S}$ is the state space,
- $\mathcal{A}$ is the action space,
- $\mathcal{P}(s' \mid s, a)$ is the transition function, i.e., the probability of reaching state s' after taking action a in state s,
- $r(s, a, s')$ is the reward function,
- ρ_0 is the initial state distribution, and
- $\mathcal{S}_f$ is the set of terminal states.

At each time step t, the environment is in state $S_t \in \mathcal{S}$. The agent, however, does not have access to the full state: the physical parameters of the payload, for instance, are not observed. Instead, it receives an observation O_t, which contains only part of the information in S_t. Based on this observation, the agent selects an action $A_t \in \mathcal{A}$ according to its policy

$$A_t \sim \pi_\zeta(\cdot \mid O_t)\,, \tag{3.2}$$

where π_ζ is a stochastic policy represented by a neural network with parameters ζ. In response:

- the environment transitions to a new state $S_{t+1} \sim \mathcal{P}(\cdot \mid S_t, A_t)$,
- it returns a scalar reward $R_{t+1} = r(S_t, A_t, S_{t+1})$, and
- the episode, which starts from an initial state $S_0 \sim \rho_0$, lasts until a terminal state in $\mathcal{S}_f$ is reached or the time limit is exceeded, at time step T.

The goal of the agent is to maximize the return, i.e., the discounted cumulative reward collected from time step t until the end of the episode,

$$G_t = \sum_{\tau=t}^{T-1} \gamma^{\tau-t} R_{\tau+1}, \tag{3.3}$$

where $\gamma \in [0,\, 1]$ is the discount factor. Accordingly, training seeks the policy parameters that maximize the expected return from the initial state:

$$J(\zeta) = \mathbb{E}_{\pi_\zeta}[G_0]\,, \qquad \zeta^\star = \arg\max_\zeta J(\zeta). \tag{3.4}$$

In this thesis, $J(\zeta)$ is maximized with PPO [13]. The following subsections define the observation, the action, the reward, and the episode of the agent that solves the transportation task.

3.1.1 Observation

The observation $O_t \in \mathbb{R}^{35}$ consists of

$$O_t = \left[p^T,\ \eta^T,\ v^T,\ \omega^T,\ (w_i - p)^T,\ \Delta_2^T, \ldots, \Delta_7^T,\ l_1,\ l_2\right]^T, \tag{3.5}$$

where

- $p \in \mathbb{R}^3$ is the position of the drone,
- $\eta = [\phi,\, \theta,\, \psi]^T$ contains the roll, pitch, and yaw angles of the drone,
- $v \in \mathbb{R}^3$ and $\omega \in \mathbb{R}^3$ are the linear and angular velocities of the drone,
- $w_i - p \in \mathbb{R}^3$ is the position of the active waypoint w_i, defined in the *Reward* subsection, relative to the drone,
- $\Delta_j = p_{\text{payload}} - p_{s_j} \in \mathbb{R}^3$ is the position of the payload relative to the j-th segment of the manipulator, $j = 2, \ldots, 7$, and
- l_1, l_2 are the normalized lengths of the two tendons.

All vector quantities are expressed in the world frame.

3.1.2 Action

The action $A_t = [a_1, \ldots, a_6]^T \in [-1,\, 1]^6$ consists of

- four normalized motor commands $a_1, \ldots, a_4$, and

- two tendon commands a_5 and a_6.

To simplify the learning of the grasping motion, the tendons are not controlled directly by the agent. Instead, they are actuated as follows:

- only the magnitude of the closing motion is learned, from a_5:

$$\mu = \frac{a_5 + 1}{2} \in [0,\, 1], \tag{3.6}$$

- the curling direction $\sigma \in \{-1,\, 1\}$ is determined from the geometry, namely from the side of the payload relative to the second segment of the manipulator, expressed in the body frame of the drone,
- the two tendons are actuated symmetrically with the commands $\sigma\mu$ and $-\sigma\mu$, while a_6 is not used, and
- the tendons are actuated only after the payload has come within $R_{\text{payload}} + 0.02\,\text{m}$ of the second segment of the manipulator and the grasp waypoint has been activated; otherwise, they are relaxed.

3.1.3 Reward

The reward at each time step is composed of three parts:

$$R_{t+1} = r_{\text{shape}} + r_{\text{bonus}} + r_{\text{term}}, \tag{3.7}$$

where

- r_{shape} is a dense shaping term,
- r_{bonus} contains sparse bonuses for completing the subtasks, and
- r_{term} is a penalty upon termination.

Both r_{shape} and r_{bonus} are built around a sequence of waypoints, of which exactly one, w_i, is active at a time. The waypoints follow each other in the order

1. take-off,
2. approach above the payload,
3. grasp pose, computed from the position and the radius of the payload, and
4. goal position.

The active waypoint w_i is considered reached if the horizontal and vertical position errors of the drone satisfy

$$e_{xy} = \|p_{xy} - w_{i,xy}\| < \varepsilon_{\text{wp}}, \qquad e_z = |p_z - w_{i,z}| < \frac{\varepsilon_{\text{wp}}}{5}, \tag{3.8}$$

where $\varepsilon_{\text{wp}} = 0.1\,\text{m}$ is the waypoint tolerance. Reaching a waypoint activates the next one, except for the grasp pose: there, the next waypoint is activated only once the payload is grasped, as defined below.

To measure how tightly the manipulator is wrapped around the payload, the cumulative segment distance is defined as

$$d_{\text{seg}} = \left\| \left[\|\Delta_2\| - R_{\text{payload}} \quad \cdots \quad \|\Delta_7\| - R_{\text{payload}} \right] \right\|, \tag{3.9}$$

i.e., the norm of the gaps between the surface of the cylinder and the manipulator segments. The payload is considered grasped when $d_{\text{seg}} < 0.1\,\text{m}$.

The shaping term, applied at every time step, is

$$r_{\text{shape}} = -e_z - 0.1\, e_{xy} - 0.03\, \left\| [\phi,\, \theta]^T \right\| - 0.06\, \|\omega\| - 0.03\, d_{\text{seg}}, \tag{3.10}$$

where e_{xy} and e_z are the errors with respect to the active waypoint in (3.8), and d_{seg} is the cumulative distance of the manipulator segments from the surface of the payload cylinder, defined in (3.9). It drives the drone toward the active waypoint, penalizes tilting and rapid rotations, and encourages the manipulator to stay close to the payload.

The sparse bonuses r_{bonus} depend on the active waypoint and are summarized in Table 3.1. Finally, the termination penalty is $r_{\text{term}} = -100$ if the episode is terminated, as described in the *Episode* subsection, and $r_{\text{term}} = 0$ otherwise.

Active waypoint	Condition	Bonus
Take-off	waypoint reached	+20
Approach	waypoint reached	+20
Grasp pose	waypoint reached	+0.1
	$d_{\text{seg}} < 0.1\,\text{m}$ (payload grasped)	+5
Goal	$d_{\text{seg}} < 0.15\,\text{m}$ (payload carried)	+3
	waypoint reached	+3

Table 3.1: Sparse reward bonuses r_{bonus} depending on the active waypoint.

3.1.4 Episode

At the beginning of each episode, the drone is placed level and at rest at $[0,\, 0,\, 0.5]^T\,\text{m}$, with a random yaw angle, and the payload and the goal are generated randomly. An episode is terminated if

- the drone crashes, i.e., $p_z < 0$,
- the drone flips over, i.e., $|\phi| > \pi/2$ or $|\theta| > \pi/2$, or
- the payload is lost, i.e., its distance from the second segment of the manipulator exceeds 1 m while the goal waypoint is active.

Otherwise, the episode is truncated after 10 s.

3.1.5 Flat Agent

In the rest of this thesis, the agent defined by the observation (3.5), the action of the *Action* subsection, the reward (3.7), and the episode of the *Episode* subsection is referred to as the *flat agent*, since it solves the whole transportation task with a single, non-hierarchical policy π_ζ, directly commanding the motors of the drone.

3.2 Curriculum Learning

Having introduced the flat agent, this section presents curriculum learning (CL). The main limitation of the flat agent is its slow training: even though the task itself has a relatively simple structure, the agent learns the desired behavior only very slowly.

CL [11] offers a solution to this problem. Instead of training the agent directly on the full task, a sequence of tasks of increasing difficulty is defined. Training starts with an easy task that the agent can learn quickly, and the task is made progressively harder, with each stage starting from the policy learned in the previous one. Since the agent always builds on the skills acquired in the earlier stages, it reaches the desired behavior considerably faster than when trained on the full task from scratch.

3.2.1 Formulation

Following [11], each task of the curriculum is an MDP of the form (3.1):

$$m_k = (\mathcal{S},\, \mathcal{A},\, \mathcal{P}_k,\, r_k,\, \rho_{0,k},\, \mathcal{S}_{f,k})\,, \qquad k = 1, \ldots, n. \tag{3.11}$$

The last task m_n is the target task, i.e., the full transportation task of the flat agent. Since the observation and action spaces are identical in all tasks, the same policy network π_ζ can be used throughout the curriculum. The tasks, however, may differ in all other elements of the MDP:

- the transition function $\mathcal{P}_k$, since the ranges from which the mass and the radius of the payload are sampled change the dynamics of the system,
- the reward function r_k, since the terms related to grasping and carrying the payload are only present once grasping is required,
- the initial state distribution $\rho_{0,k}$, through the sampling range of the goal position, and
- the set of terminal states $\mathcal{S}_{f,k}$, since losing the payload terminates the episode only in the target task.

The curriculum applied in this thesis is a task-level sequence curriculum, i.e., an ordered list of tasks

$$C = [\, m_1,\, m_2,\, \ldots,\, m_n \,], \tag{3.12}$$

where the intermediate tasks $m_1, \ldots, m_{n-1}$ are simplified versions of the target task m_n. Training starts with task m_1, and the policy parameters ζ are always carried over when the task changes. The current task is selected adaptively, based on the return G_0 in (3.3), averaged over the last N training episodes, denoted by $\bar{G}_0$:

$$m_k \to \begin{cases} m_{k+1}, & \text{if } \bar{G}_0 \geq \delta_{\text{adv}} \text{ and } k < n, \\ m_{k-1}, & \text{if } \bar{G}_0 \leq \delta_{\text{ret}} \text{ and } k > 1, \\ m_k, & \text{otherwise,} \end{cases} \tag{3.13}$$

where δ_{adv} and δ_{ret} are the thresholds for advancing to the next task and retreating to the previous one, respectively. The retreat rule prevents the agent from remaining in a task that turned out to be too difficult, e.g., due to forgetting previously learned skills.

3.2.2 Training with and without CL

This subsection presents the training of the flat agent on the task described in the *Task Description* section. Training with CL is compared to training without CL, i.e., directly on the target task.

The curriculum of the flat agent consists of $n = 4$ tasks, summarized in Table 3.2. The parameters of the task switching rule (3.13) are given in Table 3.3.

Task	Description	Grasping	m_{payload} [kg]	R_{payload} [m]	$x_{\text{goal}}, y_{\text{goal}}$ [m]
m_1	Waypoint following	no	–	–	$[-1.5, 1.5]$
m_2	Grasping a nearly fixed payload	yes	0.05	[0.02, 0.025]	$[-1, 1]$
m_3	Grasping a randomized payload	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1, 1]$
m_4	Full task	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1.5, 1.5]$

Table 3.2: Tasks of the curriculum of the flat agent.

In tasks m_1–m_3, the approach waypoint above the payload is omitted, and losing the payload does not terminate the episode. Both are introduced only in the target task m_4.

Parameter	Symbol	Value
Number of tasks	n	4
Averaging window	N	100 episodes
Advancing threshold	δ_{adv}	1000
Retreating threshold	δ_{ret}	-1000

Table 3.3: Parameters of the task switching rule.

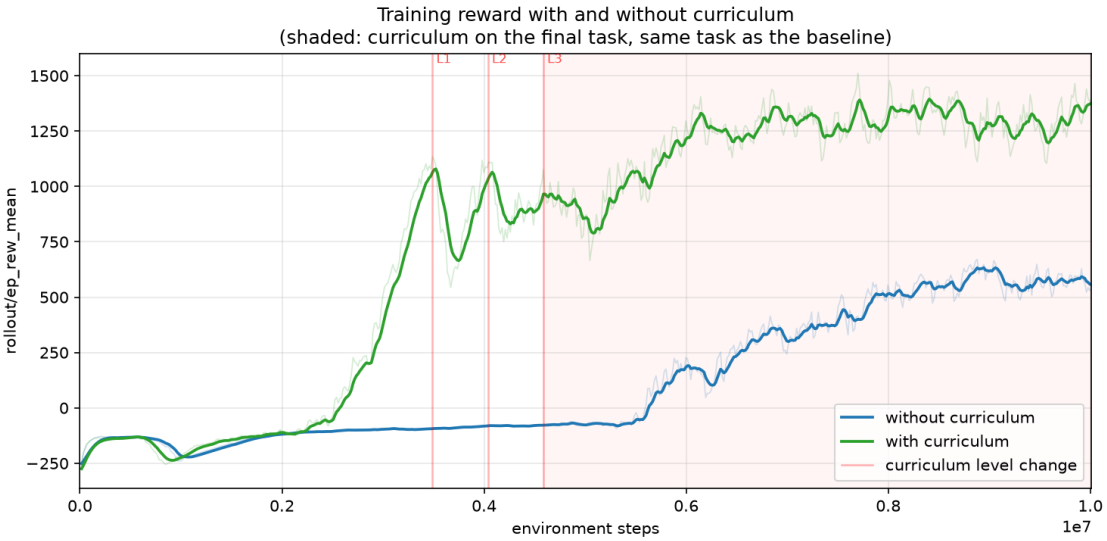


Figure 3.1: !!!NEED MORE SEEDS TO CL AND WITHOUT CL WITH COEFF 0.001. THE EXPECTED RESULT IS WITH CL IT'S CONVERGES FASTER!!!Training of the flat agent without CL (blue) and with CL (green). The horizontal axis shows the number of training steps, and the vertical axis shows the episodic return, i.e., the sum of the rewards (3.7) over an episode. The vertical lines mark the switches between the tasks of the curriculum. The results are obtained from training with three random seeds.

As can be seen in Figure 3.1, the flat agent learns the task considerably faster with CL than without it. The vertical lines indicate the time steps at which the agent advanced to the next task of the curriculum. During training, the agent only ever advanced to the next task, i.e., the retreat rule in (3.13) was never triggered. Since CL proved to be effective in this example, all policies in the remainder of this thesis are trained with CL.

3.3 Hierarchical Reinforcement Learning

As shown in the previous section, combining the flat agent with CL results in faster learning. Nevertheless, the fact that the flat agent issues RPM commands directly to the drone can lead to several problems.

First, the agent has to learn not only how to solve the task but also how to stabilize the drone, which slows down learning. Second, since the agent's actions are direct RPM commands, the learned policy may not transfer well to the real system. The real rotors may not be able to follow commands with such a high bandwidth, and due to unmodeled phenomena (e.g., motor dynamics and control delays), the motors may not track the commanded revolutions per minute (RPMs) accurately.

To address these issues, a hierarchical reinforcement learning (HRL) approach [6] is proposed, in which two agents work together: the low-level agent tracks a reference velocity and yaw angle, while the high-level agent plans the motion by providing these references.

3.3.1 Formulation

Following [6], the applied hierarchy is a feudal one: a high-level policy sets subgoals for a low-level policy, which in turn determines how these subgoals are achieved. In this thesis, the subgoal is a reference velocity and a reference yaw angle of the drone. Both levels are formulated as MDPs of the form (3.1), and the hierarchy consists of

- a high-level policy π_ζ^{H} with parameters ζ, which, similarly to the flat agent, solves the transportation task: based on the observation O_t^{H}, it selects the references and the tendon commands,
$$A_t^{\mathrm{H}} \sim \pi_\zeta^{\mathrm{H}}\big(\cdot \mid O_t^{\mathrm{H}}\big)\,, \tag{3.14}$$
- a low-level policy π_ν^{L} with parameters ν, which computes the motor commands based on its own observation O_t^{L}, containing the references given by the high level:
$$A_t^{\mathrm{L}} \sim \pi_\nu^{\mathrm{L}}\big(\cdot \mid O_t^{\mathrm{L}}\big)\,. \tag{3.15}$$

In the following, the quantities of the high-level and the low-level policy are marked with the superscripts H and L, respectively. The interaction of the two policies with each other and with the environment is shown in Figure 3.2.

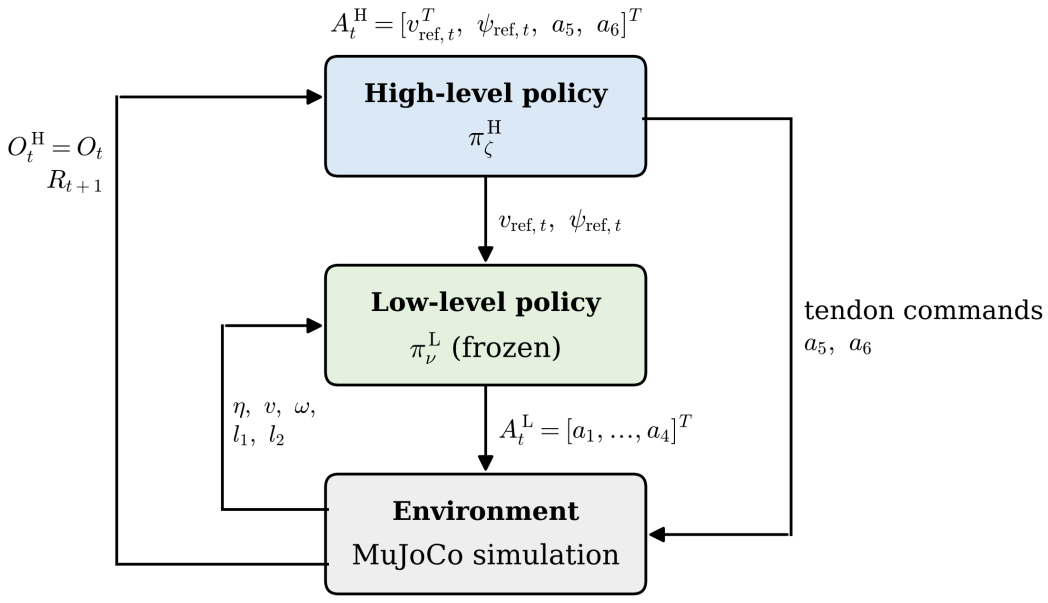


Figure 3.2: Structure of the hierarchical agent. The high-level policy π_ζ^{H} receives the same observation and reward as the flat agent, and provides the reference velocity and yaw angle to the frozen low-level policy π_ν^{L}, as well as the tendon commands to the environment. The low-level policy computes the motor commands based on the state of the drone and the references.

The two policies are trained separately. First, the low-level policy is trained on a velocity tracking task, and then its parameters ν are frozen. Afterwards, the high-level policy is trained on the transportation task, while the frozen low-level policy executes its references. Since the low-level

policy handles the stabilization of the drone, the high-level policy only has to learn *where* the drone should move and *when* the manipulator should close. Moreover, as the interface between the two levels is a velocity and yaw reference, the high-level policy can be replaced by any other planner that provides such references, e.g., the model predictive controller presented in the *Model Predictive Control* chapter.

The observation, action, reward, and episode of the low-level policy are defined in the following subsections, similarly to those of the flat agent.

3.3.2 Low-Level Observation

The observation $O_t^{\mathrm{L}} \in \mathbb{R}^{15}$ of the low-level policy consists of

$$O_t^{\mathrm{L}} = \left[\eta^T,\ v^T,\ \omega^T,\ v_{\mathrm{ref}}^T,\ \psi_{\mathrm{ref}},\ l_1,\ l_2\right]^T, \tag{3.16}$$

where

- $\eta = [\phi,\ \theta,\ \psi]^T$ contains the roll, pitch, and yaw angles of the drone,
- $v \in \mathbb{R}^3$ and $\omega \in \mathbb{R}^3$ are the linear and angular velocities of the drone,
- $v_{\mathrm{ref}} \in \mathbb{R}^3$ is the reference velocity,
- ψ_{ref} is the reference yaw angle, and
- l_1, l_2 are the normalized lengths of the two tendons.

All vector quantities are expressed in the world frame. In contrast to the flat agent, the position of the drone is not part of the observation, since velocity tracking does not depend on it.

3.3.3 Low-Level Action

The action $A_t^{\mathrm{L}} = [a_1, \ldots, a_4]^T \in [-1,\ 1]^4$ consists of the four normalized motor commands, as in the action of the flat agent. The tendons are not controlled by the low-level policy; instead, they are set by the environment:

- if a payload is carried, the hook is fully closed toward the side of the payload, and
- otherwise, the tendons are relaxed.

3.3.4 Low-Level Reward

The reward of the low-level policy at each time step is composed of three parts:

$$R_{t+1}^{\mathrm{L}} = r_{\mathrm{shape}}^{\mathrm{L}} + r_{\mathrm{bonus}}^{\mathrm{L}} + r_{\mathrm{term}}^{\mathrm{L}}, \tag{3.17}$$

where

- $r_{\mathrm{shape}}^{\mathrm{L}}$ is a dense shaping term,
- $r_{\mathrm{bonus}}^{\mathrm{L}}$ is a sparse bonus for accurate tracking, and
- $r_{\mathrm{term}}^{\mathrm{L}}$ is a penalty upon termination.

The shaping term, applied at every time step, is

$$r_{\mathrm{shape}}^{\mathrm{L}} = -e_v - e_\psi - 0.1\left(|\phi| + |\theta|\right), \tag{3.18}$$

where

- $e_v = \|v - v_{\text{ref}}\|$ is the velocity tracking error, and
- $e_\psi = |\psi - \psi_{\text{ref}}|$ is the yaw error.

It drives the drone toward the reference velocity and yaw angle, and penalizes tilting. The sparse bonus $r^{\text{L}}_{\text{bonus}}$ rewards accurate velocity tracking and is summarized in Table 3.4. Finally, the termination penalty is $r^{\text{L}}_{\text{term}} = -100$ if the episode is terminated, as described in the *Low-Level Episode* subsection, and $r^{\text{L}}_{\text{term}} = 0$ otherwise.

Condition	Bonus
$e_v < 0.025\,\text{m/s}$ (accurate velocity tracking)	+0.5
otherwise	0

Table 3.4: Sparse reward bonus $r^{\text{L}}_{\text{bonus}}$ of the low-level policy.

3.3.5 Low-Level Episode

At the beginning of each episode, the drone is placed level and at rest at $[0,\, 0,\, 12.8]^T$ m, i.e., high enough above the ground to avoid collisions while tracking the reference, with a random yaw angle in $[-\pi,\, \pi)$. The reference yaw angle is set to $\psi_{\text{ref}} = 0$, so that the policy also learns to turn toward the reference. The reference velocity is sampled from a mixture of uniform distributions, so that both hovering and fast motions are sufficiently represented:

$$v_{\text{ref}} \sim \begin{cases} \mathcal{U}\big([-0.1,\, 0.1]^3\big)\ \text{m/s}, & \text{with probability } 0.4, \\ \mathcal{U}\big([-0.5,\, 0.5]^3\big)\ \text{m/s}, & \text{with probability } 0.3, \\ \mathcal{U}\big([-1,\, 1]^3\big)\ \text{m/s}, & \text{with probability } 0.3. \end{cases} \tag{3.19}$$

To make the policy robust to the payload, in about half of the episodes a payload is attached to the drone, already grasped in the closed hook, with

$$m_{\text{payload}} \sim \mathcal{U}(0.05,\, 0.3)\,\text{kg}, \qquad R_{\text{payload}} \sim \mathcal{U}(0.02,\, 0.04)\,\text{m}. \tag{3.20}$$

An episode is terminated if

- the drone flips over, i.e., $|\phi| > \pi/2$ or $|\theta| > \pi/2$.

Otherwise, the episode is truncated after 10 s.

3.3.6 Low-Level Agent Results

As stated in the *Curriculum Learning* section, the low-level policy is also trained with a curriculum, following the formulation given in (3.11)–(3.13). Its tasks, denoted by m^{L}_k, are summarized in Table 3.5, and the parameters of the task switching rule are given in Table 3.6.

Task	Description	Random yaw	Payload	m_{payload} [kg]	R_{payload} [m]
m^{L}_1	Velocity tracking without payload	no	no	–	–
m^{L}_2	Velocity tracking with a nearly fixed payload	yes	yes	0.05	[0.02, 0.03]
m^{L}_3	Full task	yes	yes	[0.01, 0.25]	[0.02, 0.04]

Table 3.5: Tasks of the curriculum of the low-level policy.

In tasks m^{L}_2 and m^{L}_3, the payload is attached to the drone in about half of the episodes, as described above.

Parameter	Symbol	Value
Number of tasks	n	3
Averaging window	N	20 episodes
Advancing threshold	δ_{adv}	110
Retreating threshold	δ_{ret}	-1000

Table 3.6: Parameters of the task switching rule of the low-level policy.

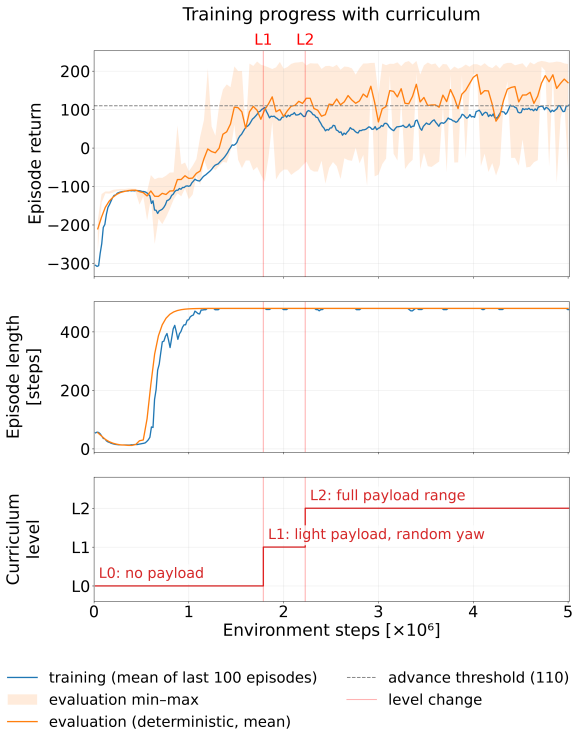


Figure 3.3: Training of the low-level policy: episodic return (3.17) (top), episode length (middle), and curriculum task (bottom) versus environment steps. Blue: mean over 8 training environments; orange and shaded area: mean and min–max range in the deterministic evaluation environment. Light red vertical lines mark the task switches.

As can be seen in Figure 3.3, the training of the low-level policy converges: by the end of the training, the mean training return stabilizes at around 100, while the evaluation return reaches about 150–180. The episode length quickly reaches its maximum of 480 steps, i.e., 10 s at 48 Hz, which means that the drone no longer flips over. The effect of the curriculum is also visible: after the switch to the full task m_3^{L}, the return drops, since the policy has to learn velocity tracking under more difficult conditions, with payloads of widely varying mass and radius. By the end of the training, however, the policy successfully adapts to these conditions.

To evaluate the trained low-level policy, 200 test runs were performed with a piecewise constant reference velocity profile of 12 s, in which the reference changes at $t = 1$, 3, 5, 6.5, 8.5, and 10 s. In each run, the initial yaw angle of the drone is random, and in 93 runs a payload with random mass and radius is attached to the drone, while the remaining 107 runs are performed without payload. Note that the evaluation differs from the training in two respects: the reference velocity changes during the episode instead of being constant, and the episode is longer than the 10 s used in training. The results are shown in Figures 3.4–3.6.

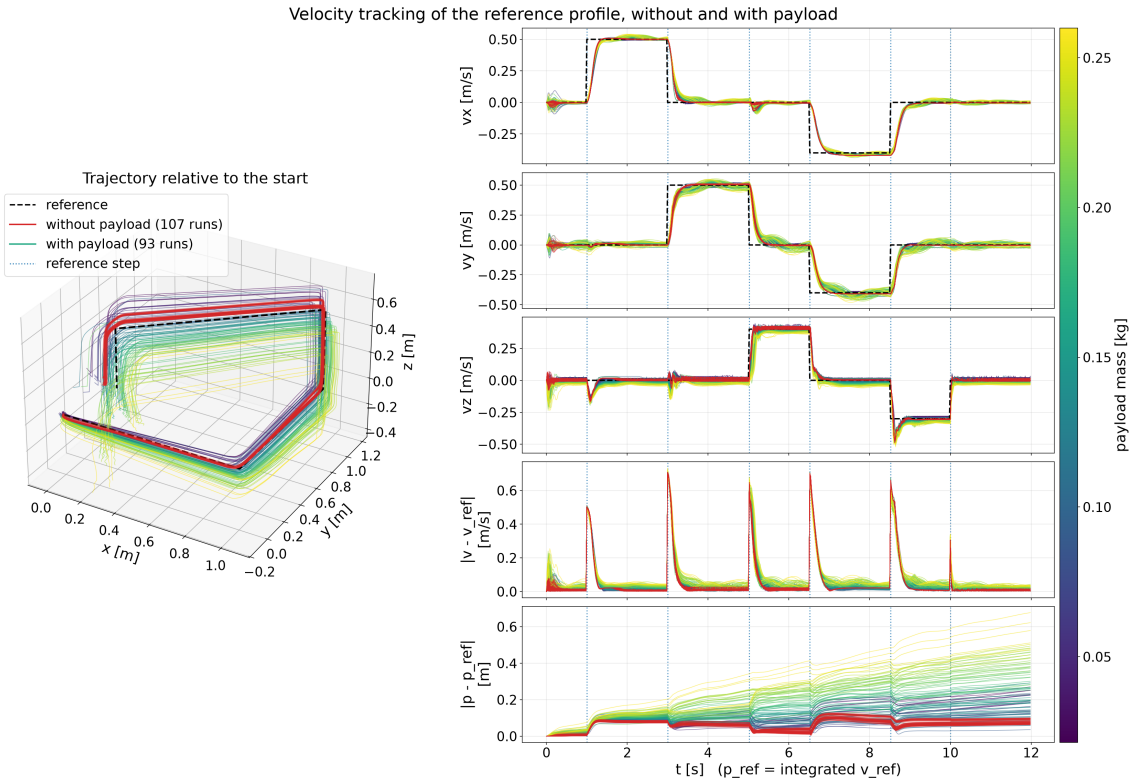


Figure 3.4: Velocity tracking of the low-level policy over 200 test runs. Left: trajectory relative to the starting position. Right, from top to bottom: velocity components v_x, v_y, v_z, velocity error norm $\|v - v_{\text{ref}}\|$, and position error $\|p - p_{\text{ref}}\|$ with respect to the integrated reference velocity. Red: runs without payload; other colors: runs with payload, colored by the payload mass. Black dashed line: reference; dotted vertical lines: reference steps.

As can be seen in Figure 3.4, the low-level policy tracks the reference velocity well in all runs. After each reference step, the velocity settles within about 0.3–0.5 s, and the large peaks of the velocity error appear only at the steps themselves, where the reference changes discontinuously. A step in the horizontal velocity also causes a short transient in the vertical velocity, since the drone has to tilt to accelerate horizontally, which temporarily reduces the vertical component of the thrust. Although the policy was trained only with constant references, it also tracks a changing reference reliably, which is essential for its use under the high-level policy.

The effect of the payload is most visible in the position error. Since the low-level policy only tracks the velocity, small steady-state velocity errors accumulate in the position. Without payload, the position error remains around 0.1 m over the whole run, while for the heaviest payloads it grows up to about 0.5–0.7 m by the end of the run. This drift, however, does not affect the overall task, since the position is controlled by the high-level policy.

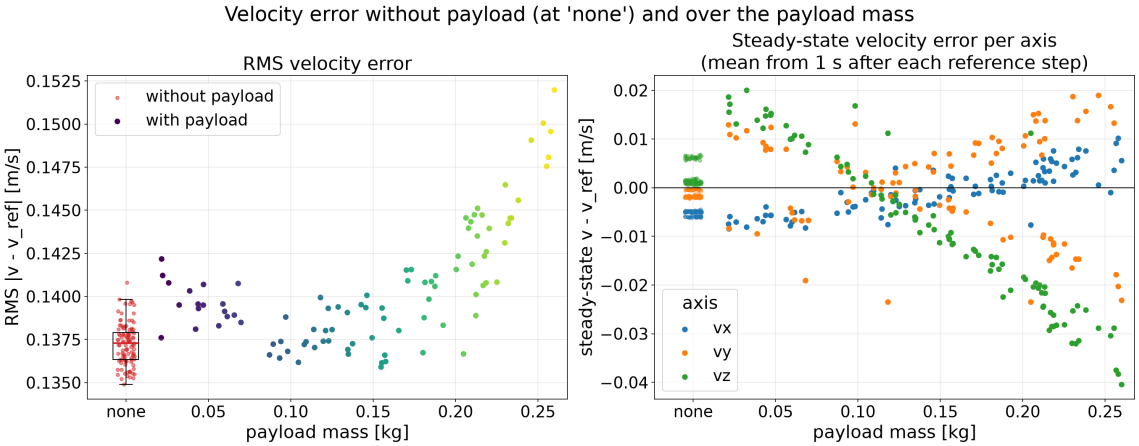


Figure 3.5: Velocity tracking error of the low-level policy as a function of the payload mass; "none" denotes the runs without payload. Left: RMS velocity error over the whole run. Right: steady-state velocity error per axis, averaged from 1 s after each reference step.

Figure 3.5 quantifies the effect of the payload mass. Without payload, the RMS velocity error is between 0.135 and 0.141 m/s. With payload, it increases with the payload mass, up to about 0.152 m/s for the heaviest payloads. Since the RMS error is computed over the whole run, it is dominated by the transients after the reference steps; the increase caused by the payload is therefore moderate, about 10 %. The steady-state error shows the cause of the position drift more clearly. Without payload, it stays below 0.01 m/s on every axis, whereas with increasing payload mass, the vertical error becomes increasingly negative, reaching about −0.04 m/s for the heaviest payloads. In other words, with a heavy payload, the drone sinks slightly faster or climbs slightly slower than commanded, since the policy does not fully compensate for the additional weight. Nevertheless, the steady-state error remains small compared to the reference velocities of 0.3–0.5 m/s.



Figure 3.6: Yaw tracking of the low-level policy over 200 test runs, with a random initial yaw angle and a reference yaw angle of $\psi_{\text{ref}} = 0$. Red: runs without payload; other colors: runs with payload, colored by the payload mass. The gray band indicates ±5°; dotted vertical lines mark the steps of the reference velocity.

Finally, Figure 3.6 shows the yaw tracking. Starting from a random initial yaw angle in the range of $\pm 180°$, the drone turns to the reference yaw angle within about 0.3 s in all runs, and then keeps the yaw error within $\pm 5°$. The steps of the reference velocity cause short yaw disturbances, the largest ones, of up to about $35°$, occurring at the vertical velocity steps at $t = 5$ and 8.5 s, where the collective thrust changes abruptly. These disturbances are somewhat larger with heavier payloads, but they decay within about 0.3 s in every run. Overall, the evaluation shows that the low-level policy provides fast and robust velocity and yaw tracking, both without and with payloads of various masses.

3.3.7 High-Level Policy

The high-level policy is trained on the same transportation task as the flat agent, and its formulation differs from that of the flat agent only in the action.

The observation of the high-level policy is identical to that of the flat agent, i.e., $O_t^{\mathrm{H}} = O_t$, given in (3.5).

The action A_t^{H} of the high-level policy consists of

- the reference velocity $v_{\mathrm{ref},t} \in \mathbb{R}^3$ and the reference yaw angle $\psi_{\mathrm{ref},t}$, which are passed to the low-level policy in (3.16), and
- the two tendon commands a_5 and a_6, which are processed in the same way as in the action of the flat agent, see (3.6).

The reward, the initial state distribution, and the termination conditions are the same as for the flat agent.

3.3.8 High-Level Agent Results

The high-level policy is also trained with CL, using the same curriculum tasks and switching parameters as the flat agent, given in Tables 3.2 and 3.3. Since the high-level agent is trained in the same environment, with the same reward function, observation space, and curriculum switching conditions as the flat agent, its training can be directly compared to that of the flat agent shown in Figure 3.1.

!!LEARNING RESULTS LIKE 3.3!!

!!COMPARE TO CL FLAT MODEL (3seed vs 3seed), NEED 3seed ON RL ADAPTIVE DIRECTOR AND MAKE A COMPREHANSION TO THE CL FLAT MODEL WITH THE SAME CL SETTINGS. THE EXPECTED RESULT IS THE DIRECTOR CONVERGES FASTER BECAUSE IT DOESNT NEED TO LEARN TO STABLIZE THE DRONE. MAKE THE PLOT AND ANALYSE THE RESULTS!!

The detailed validation of the high-level policy is presented in the *Comprehension* chapter, where it is compared to the model predictive controller, and the RL agent with model predictive safety filter.

Chapter 4

Model Predictive Control

This chapter presents model predictive control (MPC), another method capable of solving the payload transportation task. MPC is widely applied to drone–manipulator systems, for example, for payload transportation with a hook [2] or for opening doors [20], which makes it a natural choice for this work as well.

MPC has several advantages. First, constraints on the states and the control inputs, such as velocity limits, can be handled explicitly in the optimization problem, which leads to safe and predictable behavior. Second, since MPC relies on a model of the system rather than on previously collected experience, it can also perform well in situations that were not encountered during its design [12]. Third, the smoothness of the control inputs can be enforced directly, e.g., by penalizing their magnitude and their changes in the cost function.

Nevertheless, MPC also has drawbacks. Most importantly, it requires a mathematical model that describes the system accurately, and its performance is sensitive to modeling errors. As mentioned in the *System Modeling* chapter, the identification of the drone–manipulator system is far from trivial. Therefore, a data-driven approach is applied, in which the dynamics are modeled by a feedforward residual neural network (ResNet) [3]. Instead of the full system, the identified model describes the closed-loop behavior of the drone controlled by the low-level policy presented in the previous chapter, i.e., how the drone responds to a given reference velocity.

As a result, MPC fits naturally into the hierarchical architecture presented in the *Hierarchical Reinforcement Learning* section: it replaces the high-level policy and provides the reference velocity for the low-level policy, based on the identified closed-loop dynamics of the low level. The first part of this chapter introduces the formulation of the MPC, while the second part presents the identification procedure.

4.1 Model Predictive Control Formulation

!!! THIS SUBSECTION IS NOT GOOD, PLEASE WHEN DO THE FORMUALTION (THE OPTIMIZATION PROBLEM) FOMUALTE OUR MPC PROBLEM AND DEFINE THE ELEMENTS, DEFINE THE STATE VECTOR ETC!!!

Throughout this chapter, the terminology of [4] is followed. MPC is based on numerical optimization: at each control step, a cost function is minimized subject to constraints, the resulting optimal control input is applied to the system, and the procedure is repeated from the new state of the system.

Consider a discrete-time system with state $x \in \mathbb{R}^{n_x}$ and control input $u \in \mathbb{R}^{n_u}$, whose dynamics are described by the model

$$x_{k+1} = f(x_k, u_k). \tag{4.1}$$

At each control step, MPC predicts the behavior of the system over a finite horizon of H steps using (4.1), and determines the control input sequence that minimizes a cost function over this

horizon. The resulting finite-horizon optimal control problem is

$$\min_{u_0,\ldots,u_{H-1}} \quad \mathcal{J} = \sum_{k=0}^{H-1} \ell(x_k,\, u_k) + \ell_f(x_H) \tag{4.2a}$$

$$\text{s.t.} \quad x_0 = \bar{x}, \tag{4.2b}$$

$$x_{k+1} = f(x_k,\, u_k)\,, \qquad k = 0, \ldots, H-1, \tag{4.2c}$$

$$x_k \in \mathcal{X}, \qquad k = 1, \ldots, H, \tag{4.2d}$$

$$u_k \in \mathcal{U}, \qquad k = 0, \ldots, H-1, \tag{4.2e}$$

where

- (4.2a) is the cost function $\mathcal{J}$, composed of the path cost ℓ summed over the prediction steps k of the horizon and the terminal cost ℓ_f,
- (4.2b) is the initial state constraint, where $\bar{x}$ is the current (measured or estimated) state of the system, which serves as the initial condition of the prediction,
- (4.2c) is the dynamic constraint, which enforces that the predicted states follow the model (4.1),
- (4.2d) is the state constraint, where $\mathcal{X}$ is the set of admissible states, and
- (4.2e) is the input constraint, where $\mathcal{U}$ is the set of admissible inputs.

Since the system model used in this thesis is a neural network, (4.2) is a nonconvex optimization problem, for which the solver can only be expected to find a locally optimal solution [4].

Solving (4.2) yields the optimal input sequence $u_0^\star, \ldots, u_{H-1}^\star$, of which only the first element $u_0^\star$ is applied to the system. At the next control step, (4.2) is solved again with $\bar{x}$ set to the new state of the system, i.e., the horizon is shifted by one step. This receding horizon principle introduces feedback into the optimization: since each optimization starts from the actual state of the system, modeling errors and disturbances are continuously compensated.

Bibliography

[1] Péter Antal, Tamás Péni, and Roland Tóth. Modelling, identification and geometric control of autonomous quadcopters for agile maneuvering. *Repüléstudományi Közlemények*, 35(1): 141–160, 2023. DOI: 10.32560/rk.2023.1.11.

[2] Péter Antal, Andrea Carron, Melanie Zeilinger, Roland Tóth, and Tamás Péni. Robust adaptive predictive control for hook-based aerial transportation between moving platforms. *IEEE Robotics and Automation Letters*, 11(11):12352–12359, 2026. DOI: 10.1109/LRA.2026.3730216.

[3] Pierre Clément Blaud, Philippe Chevrel, Fabien Claveau, Pierrick Haurant, and Anthony Mouraud. Resnet and polynet based identification and (mpc) control of dynamical systems: A promising way. *IEEE Access*, 11:20657–20672, 2023. DOI: 10.1109/ACCESS.2022.3196920.

[4] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[5] Joshua Fishman and Luca Carlone. Control and trajectory optimization for soft aerial manipulation. In *2021 IEEE Aerospace Conference (50100)*, pages 1–17, 2021. DOI: 10.1109/AERO50100.2021.9438502.

[6] Martin Klissarov, Akhil Bagaria, Ziyan Luo, George Konidaris, Doina Precup, and Marlos C. Machado. Discovering temporal structure: An overview of hierarchical reinforcement learning, 2026. URL https://arxiv.org/abs/2506.14045.

[7] Hughes La Vigne, Guillaume Charron, Julien Rachiele-Tremblay, David Rancourt, Ben Nyberg, and Alexis Lussier Desbiens. Collecting critically endangered cliff plants using a drone-based sampling manipulator. *Scientific Reports*, 12(1):14827, 2022. ISSN 2045-2322. DOI: 10.1038/s41598-022-17679-x. URL https://doi.org/10.1038/s41598-022-17679-x.

[8] Chen Liang and Daniel Rakita. Robust and efficient mujoco-based model predictive control via web of affine spaces derivatives, 2026. URL https://arxiv.org/abs/2512.21109.

[9] Zoltán Miklós Misley. Modeling, simulation and control design for yo-yo playing with quadcopters. Bsc thesis, Budapest University of Technology and Economics, 2025.

[10] Ezeldin N. Moustafa, Mitsuhiro Kamezaki, Ahmed Khalifa, Shota Miyake, Tamon Miyake, and Shigeki Sugano. Development of a two-dof three-parallelogram aerial manipulator for pipeline crack repair using a two-tool end-effector and onboard sensor-based localization. In *2025 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1–7, 2025. DOI: 10.1109/AIM64088.2025.11175832.

[11] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *CoRR*, abs/2003.04960, 2020. DOI: 10.48550/ARXIV.2003.04960. URL https://doi.org/10.48550/arXiv.2003.04960.

[12] Rudolf Reiter, Jasper Hoffmann, Dirk Reinhardt, Florian Messerer, Katrin Baumgärtner, Shambhuraj Sawant, Joschka Bödecker, Moritz Diehl, and Sebastien Gros. Synthesis of model predictive control and reinforcement learning: Survey and classification. *Annual Reviews in Control*, 61:101045, 2026. ISSN

1367-5788. DOI: https://doi.org/10.1016/j.arcontrol.2026.101045. URL https://www.sciencedirect.com/science/article/pii/S1367578826000015.

[13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[14] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 978-0-262-03924-6. URL http://incompleteideas.net/book/the-book-2nd.html.

[15] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. DOI: 10.1109/IROS.2012.6386109.

[16] B. V. Vidyadhara, Lima Agnel Tony, Mohitvishnu S. Gadde, Shuvrangshu Jana, V. P. Varun, Aashay Anil Bhise, Suresh Sundaram, and Debasish Ghose. Design and integration of a drone based passive manipulator for capturing flying targets. *Robotica*, 40(7):2349–2364, 2022. DOI: 10.1017/S0263574721001673.

[17] Kim Peter Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021. ISSN 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2021.109597. URL https://www.sciencedirect.com/science/article/pii/S0005109821001175.

[18] Zhanchi Wang, Nikolaos M. Freris, and Xi Wei. Spirobs: Logarithmic spiral-shaped robots for versatile grasping across scales. *Device*, 3(4):100646, 2025. ISSN 2666-9986. DOI: https://doi.org/10.1016/j.device.2024.100646. URL https://www.sciencedirect.com/science/article/pii/S2666998624006033.

[19] Chao Zeng, Zhan Li, Yipeng Yang, Cunjia Liu, and Chenguang Yang. Learning-based methods for aerial manipulation: A focused review. *IEEE/ASME Transactions on Mechatronics*, 31 (2):2270–2287, 2026. DOI: 10.1109/TMECH.2025.3625915.

[20] Wen-Shuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020. URL https://api.semanticscholar.org/CorpusID:221971078.